

Appendix J

Some general purpose algorithms

This appendix describes a few algorithms that are used in the text.

J.1 Gaussian distribution

Algorithm 36 (Gaussian distribution)

If R_1 and R_2 are two random numbers distributed uniformly over the interval $\{0, 1\}$, then X_g , given by
$$X_g = X_{\text{avg}} + \sigma \sqrt{-\ln(R_1)} \cos(2\pi R_2)$$
has a Gaussian distribution with average X_{avg} and variance σ^2

Comment *The above algorithm is but one example. It is simple, but not necessarily the fastest.*

J.4 Generate bond length

Algorithm 39 (Generate bond length with harmonic springs (3d))

function bondl	returns bond length ℓ
	assume harmonic springs.
	Spring constant k_v .
	ℓ_0 : bond length at $T = 0$
$\alpha = k_v / (k_B T)$	
$\ell_M = (\ell_0/2) * (1 + \sqrt{1 + 8/(\alpha \ell_0^2)})$	Position maximum at T
ready=.false.	
while ready == .false. do	
$\ell = \text{gauss}(\alpha, \ell_M)$	generate ℓ with a Gaussian distr.
aux = $2 * [-(\ell/\ell_M - 1) + \ln(\ell/\ell_M)]$	auxiliary quantity
if $\mathcal{R} \leq \exp(\text{aux})$ then	accepted ?
ready=.true.	
endif	
enddo	rejection step
end function	

Specific Comments (for general comments, see p. 7)

1. The bond length has the following distribution:

$$p(\ell) \propto \exp[-\beta 0.5 k_v (\ell - \ell_0)^2] d\ell \propto \ell^2 \exp[-\beta 0.5 k_v (\ell - \ell_0)^2] d\ell.$$

2. We make use of the fact that $x - 1 \geq \ln x$.

3. $\text{gauss}(\alpha, \ell_M)$ is a 1d normal distribution, see section J.1.

The distribution of bond lengths of linear, harmonic molecules in 3d is close to a Gaussian distribution and can be generated starting from the Box-Muller algorithm [66].

Consider a linear molecule that can vibrate along its axis. The force constant for the vibration is denoted by κ . For a fixed orientation, the equilibrium bond-length is denoted by ℓ_0 . We denote the inverse temperature by β . We ignore coupling between rotation and vibration.

Under these circumstances, the length-distribution of the molecule is

$$P(\ell) \sim \ell^2 \exp[-0.5\beta\kappa(\ell - \ell_0)^2] = \exp[-0.5\beta\kappa(\ell - \ell_0)^2 + 2\ln \ell]. \quad (\text{J.4.1})$$

For what follows, the normalization is unimportant. We cannot sample this distribution directly. However, we can use the rejection method. First, we determine the location ℓ_M of the maximum of $\ln P(\ell)$. We then get:

$$\beta\kappa(\ell_M - \ell_0) - 2/\ell_M = 0. \quad (\text{J.4.2})$$

Denoting $\beta\kappa$ by α we get

$$\ell_M^2 - \ell_M \ell_0 - 2/\alpha = 0, \quad (\text{J.4.3})$$

or

$$\ell_M = 0.5(\ell_0 + \sqrt{\ell_0^2 + 8/\alpha}). \quad (\text{J.4.4})$$

We can approximate Eq. (J.4.1) by a Gaussian around ℓ_0 :

$$P'(\ell) \sim \exp[-0.5\alpha(\ell - \ell_M)^2]. \quad (\text{J.4.5})$$

$P'(\ell)$ is related to $P(\ell)$ through

$$P(\ell)/P'(\ell) = \exp\left[2\ln(\ell/\ell_M) - \frac{2(\ell - \ell_M)}{\ell_M}\right] \leq 1.$$

We can therefore draw values of ℓ from $P'(\ell)$ and then reject values of ℓ if

$$\mathcal{R} > \exp\left[2\ln(\ell/\ell_M) - \frac{2(\ell - \ell_M)}{\ell_M}\right].$$

J.5 Generate bond angle

Algorithm 40 (Generate bond angle)

<pre> function bonda(xn,i) ready=.false. while ready == .false. do b = ranor dx1x2=xn(i-1)-xn(i-2) u12=dx1x2/ dx1x2 theta=acos(b*dx1x2) bu = ubb(theta) if $\mathcal{R} < \exp(-\beta u)$ then ready=.true. endif endif end function </pre>	generate bond orientation vector $\mathbf{b}$ with Boltzmann probability given by the bond-bending potential unit vector on a sphere vector $\mathbf{r}_{21} = \mathbf{r}_{i-1} - \mathbf{r}_{i-2}$ normalize vector bending angle $\theta = \arccos(\hat{\mathbf{u}}_{12} \cdot \hat{\mathbf{b}})$ bond-bending energy rejection test
---	---

Specific Comment (for general comments, see p. 7)

1. This algorithm uses a naive rejection scheme to generate a Boltzmann distribution of orientations $\hat{\mathbf{b}}$. Function **ranor** generates a random vector on a unit sphere (Algorithm 38). The function **ubb** (not specified) gives the bond-bending energy for the given angle.

J.6 Generate bond and torsion angle

Algorithm 41 (Generate bond and torsion angle)

<pre> function tors_bonda(xn,i) ready=.false. while ready == .false. do b = ranor dx1x2=xn(i-1)-xn(i-2) dx1x2=dx1x2/ dx1x2 dx2x3=xn(i-2)-xn(i-3) dx2x3=dx2x3/ dx2x3 theta=acos(b * dx1x2) ubb=ubb(theta) xx1=b × dx1x2 xx2=dx1x2 × dx2x3 [...normalize xx1 and xx2] phi=acos(xx1 * xx2) utors=utors(phi) usum=ubb+utors if $\mathcal{R} < \exp(-\beta * usum)$ then ready=.true. endif enddo end function </pre>	generate a unit vector with orientational Boltzmann distribution determined by torsion and bond-bending potentials generate random unit vector $\hat{\mathbf{b}}$ vector $\mathbf{r}_{21} = \mathbf{r}_{i-1} - \mathbf{r}_{i-2}$ normalize $\mathbf{r}_{12}$: $\hat{\mathbf{u}}_{12} \equiv \mathbf{r}_{12}/ \mathbf{r}_{12}$ vector $\mathbf{r}_{23} = \mathbf{r}_{i-2} - \mathbf{r}_{i-3}$ normalize $\mathbf{r}_{23}$: $\hat{\mathbf{u}}_{23} \equiv \mathbf{r}_{23}/ \mathbf{r}_{23}$ bending angle $\theta = \arccos(\hat{\mathbf{u}}_{12} \cdot \hat{\mathbf{b}})$ bond-bending energy cross product: $\mathbf{xx1} = \hat{\mathbf{b}} \times \hat{\mathbf{u}}_{12}$ cross product: $\mathbf{xx2} = \hat{\mathbf{u}}_{12} \times \hat{\mathbf{u}}_{23}$ torsion angle $\phi = \arccos(\mathbf{xx1} \cdot \mathbf{xx2})$ determine torsion energy rejection test
--	---

Specific Comments (for general comments, see p. 7)

1. This algorithm uses a naive rejection scheme to generate a Boltzmann distribution of orientations $\hat{\mathbf{b}}$.
2. In the literature, various definitions of the torsion angle are used.
3. The function **ranor** generates a vector uniformly on a unit sphere (Algorithm 38), and the function **ubb** (not specified) gives the bond-bending energy for the given angle θ . The function **utors** (also not specified) gives the torsion energy for the dihedral angle ϕ .